

บทที่ 5

ทรานแซกชัน

ในบทนี้จะพูดถึงเกี่ยวกับ Web Services Transaction ที่เราได้นำมาใช้ในโครงการและแสดงให้เห็นการทำงานของ WS-Transaction และการนำมาใช้งานในโครงการ

5.1 ทรานแซกชัน คืออะไร

Transaction หมายถึง ชุดลำดับของคำสั่งในการประมวลผลข้อมูลที่ถูกจัดรวมเป็นหนึ่งงาน ซึ่งคุณสมบัติ ดังนี้

1. Atomicity จะทำการประมวลผลอย่างแยกออกจากกันไม่ได้ ถ้าทำงานไม่สำเร็จทั้งหมดก็ต้องยกเลิกทั้งหมด
2. Consistency ฐานข้อมูลจะต้องอยู่ในสภาพที่ถูกต้องเสมอ ตราบใดที่โปรแกรมของผู้ใช้ไม่มีข้อผิดพลาดในทางตรรกะธุรกิจ (business logic) การประมวลผล Transaction จะต้องไม่ทำให้ข้อมูลในฐานข้อมูลผิดพลาด
3. Isolation ไม่ว่าในระบบจะมี Transaction ทำงานอยู่ในจำนวนเท่าไรก็ตาม การทำงานของ Transaction หนึ่งจะต้องทำงานร่วมกับ Transaction อื่นเสมือนว่าแต่ละ Transaction นั้นทำงานเป็นอิสระซึ่งกันและกัน
4. Durability เมื่อ Transaction หนึ่งทำงานสิ้นสุดลงแล้ว ผลของการประมวลผลของ Transaction จะต้องไม่สูญหาย ถึงแม้ว่าระบบมีเหตุขัดข้องหลังจากที่ Transaction จะเสร็จสิ้น เช่น เครื่องหยุดการทำงานเนื่องจากไฟฟ้าดับ ระบบจัดการฐานข้อมูลจะต้องรักษาสถานะของฐานข้อมูลที่ถูกต้อง

คุณสมบัติทั้ง 4 ข้อ เรียกว่ากฎ ACID เพื่อให้บรรลุตามคุณสมบัตินี้ ระบบจัดการฐานข้อมูลต้องทำงาน ดังต่อไปนี้

1. ระบบจัดการฐานข้อมูลจะทำงานในลักษณะของ Transaction โดยสามารถรับการกำหนดจากโปรแกรมระบบงานของผู้ใช้ว่า Transaction นั้นเริ่มต้นและสิ้นสุดเมื่อไร
2. เก็บรายการเปลี่ยนแปลงในฐานข้อมูล (Logging) ทุกรายการที่มีการเปลี่ยนแปลงจะต้องมีการเก็บบันทึกลงในสื่อ เช่น Hard disk เมื่อระบบมีข้อผิดพลาดเกิดขึ้น ระบบจัดการฐานข้อมูลสามารถนำข้อมูลดังกล่าวมาปรับปรุงข้อมูลให้อยู่ในสภาพที่ถูกต้องดังเดิม
3. ความคุมการทำงานพร้อมกัน (Concurrency Control) ถ้า Transaction มากกว่าหนึ่งงานจะต้องทำงานเสมือนแต่ละงานเป็นอิสระต่อกัน ระบบจัดการฐานข้อมูลจะทำการควบคุมการทำงานพร้อมกันได้ด้วยการควบคุมการเข้าถึงข้อมูลในระบบฐานข้อมูล โดยระบบจัดการฐานข้อมูล

จะต้องจัดลำดับในการประมวลผล เพื่อให้การทำงานโดยรวมเทียบเท่ากับแต่ละงานทำงานเป็นหน่วยเดียวกันตั้งแต่ต้นจนจบ

4. การแก้ไขการติดขัด (Deadlock Resolution) ในกรณีที่เกิดการแย่งกันเข้าถึงข้อมูล จนทำให้ระบบไม่สามารถทำงานต่อไปได้ ระบบจัดการฐานข้อมูลจะต้องทำการแทรกแซงและยกเลิกการทำงานของบางรายการเพื่อให้ระบบโดยรวมสามารถทำงานต่อไปได้

Specification ของ Web Services Transactions กำหนดวิธีการทำงานของ Transaction ในการติดต่อสื่อสารระหว่าง Web Services ที่เป็นผู้เรียกและ Web Services ที่เป็นผู้ให้บริการในการดึงเอา Transaction มาใช้ในการเพิ่มคุณภาพของการติดต่อสื่อสารบน Web Services

Specification ของ Web Services Transactions ได้ทำการระบุรายละเอียดสำหรับ

- Short duration, ACID transactions (WS-AtomicTransaction)
- Long running business transactions (WS-BusinessActivity)

5.2 WS-Coordination (WS-COOR)

มาตรฐานนี้จะระบุ Framework สำหรับการจัดหา protocol ซึ่งคอยประสานการทำงานของ application ที่อยู่บนระบบแบบ distributed โดย protocol สำหรับการประสานงานนั้นจะใช้สำหรับรองรับจำนวนของ application หลายๆ application โดยจะรวมทุกๆ อย่างที่จำเป็นในการติดต่อสื่อสารกันของระบบแบบ distributed

Framework ที่ถูกกำหนดไว้ใน specification นี้จะทำให้ application ที่เป็นผู้ประสานงานสามารถสร้างสภาพแวดล้อมเพื่อประกาศการกระทำในขณะนั้นให้กับ services อื่นๆ และยังเป็นการลงทะเบียนสำหรับ protocol ในการประสานงานและ framework ยังคงทำให้การดำเนินการกับ transaction ในปัจจุบัน, การไหลของงานที่ทำและระบบอื่นๆ สำหรับการประสานงานสามารถที่จะซ่อน protocol ที่เป็นผู้เจ้าของเอาไว้และยังสามารถทำงานได้บนระบบที่ต่างกัน

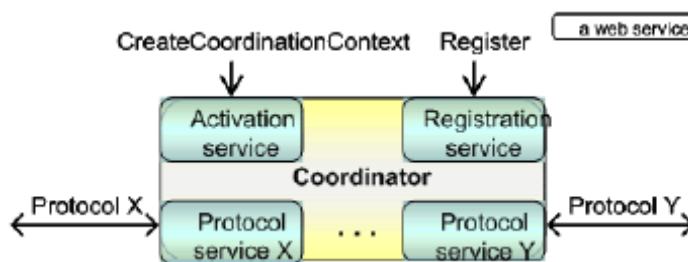
นอกจากนี้ Specification นี้ยังอธิบายเกี่ยวกับคำนิยามของโครงสร้างสภาพแวดล้อมและความต้องการของระบบในการสนับสนุนการปฏิบัติงานร่วมกันของ services ต่างๆ

5.2.1 Model

WS-Coordination ได้ทำการบรรยายถึงโครงของ Coordination Service (Coordinator) เอาไว้ซึ่งประกอบไปด้วย Service ดังนี้

- Activation Service, ซึ่งเป็นกระบวนการในการสร้าง Coordination instance (Context)
- Registration Service, ซึ่งเป็นกระบวนการในการให้ แอปพลิเคชัน register เข้ากับ coordination protocol

- Coordination Type ซึ่งเป็นเซตของ Coordination protocol



รูปที่ 5.1 เซอร์วิสที่ประกอบภายในCoordinator

แอปพลิเคชัน มีการใช้ Activation Service ในการที่จะสร้าง Coordination Context สำหรับ Activity ของตัวเอง เมื่อ Coordination Context ถูกนำไปโดย แอปพลิเคชันอื่น Coordination Context จะถูกส่งไปด้วยอะไรก็ตามที่เหมาะสมสำหรับ แอปพลิเคชันอื่นนั้น

Context จะประกอบไปด้วยข้อมูลที่สำคัญในการที่จะให้ แอปพลิเคชันอื่นนั้นสามารถที่จะเข้ามา Register สำหรับ Activity ได้โดยการกำหนดพฤติกรรมของ Coordination ที่แอปพลิเคชันนั้นต้องทำตาม

นอกจากนั้น แอปพลิเคชัน ที่ได้รับ Coordination Context ยังสามารถที่จะใช้ Registration Service กับตัวมันเอง หรืออาจจะใช้กับ Trusted Coordinator อื่นก็ได้ ในกรณีนี้ กลุ่มของ Web Service อาจจะเข้ามาสัมพันธ์กันในกระบวนการเชื่อมต่อ

5.2.2 Extensibility

โครงสร้างของ WS-Coordination มีความสามารถในการที่จะต่อขยายรวมไปถึงมีความยืดหยุ่นสำหรับการใช้งาน โดยโครงสร้างจะอนุญาตให้

- สามารถทำการตีพิมพ์ เผยแพร่ Coordination Protocols ใหม่ๆ ได้
- Protocol ที่ถูกเลือกมาจาก Coordination Type และคำจำกัดความของส่วนที่ต่อขยายสามารถทำการเพิ่มเข้าไปใน Protocol และ message flows

ส่วนที่ทำการต่อขยายเข้ามาสามารถใช้ในการแลกเปลี่ยนข้อมูลได้บน message flow ที่ถูกจำกัดความไว้ใน WS-Coordination ซึ่งนำไปสู่ความต้องการในการแลกเปลี่ยนข้อมูลใน Isolation-level หรือใน Business-Level Coordination protocols ข้อมูลที่แลกเปลี่ยนกันสามารถทำการบันทึกไว้ได้เพื่อทำการตรวจสอบในภายหลัง

5.2.3 Coordination Context

Coordination Context คือ ชนิดของ Context ชนิดหนึ่งที่ใช้ในการส่งข้อมูลของ Coordination ให้กับ Participants อื่นๆทุกตัวที่เกี่ยวข้องใน Coordination Service (Coordinator) ซึ่งตัว Coordination Context จะถูกใส่รวมไว้กับ แอปพลิเคชัน (Conveying a context-flowing the context)

CoordinationContext จะให้ความสามารถในการ coordination registration service, Coordination Type และ relevant extensions ตัวอย่างของ *CoordinationContext*

```
<?xml version="1.0" encoding="utf-8"?>

<S:Envelope xmlns:S="http://www.w3.org/2003/05/soap-envelope">

<S:Header>

... <wscoor:CoordinationContext

xmlns:wsa="http://schemas.xmlsoap.org/ws/2004/08/addressing"

xmlns:wscoor="http://schemas.xmlsoap.org/ws/2004/10/wscoor"

xmlns:myApp="http://fabrikam123.com/myApp"

S:mustUnderstand="true">

<wscoor:Identifier>

http://Fabrikam123.com/SS/1234

</wscoor:Identifier>

<wscoor:Expires>3000</wscoor:Expires>

<wscoor:CoordinationType>

http://schemas.xmlsoap.org/ws/2004/10/wsat

</wscoor:CoordinationType>

<wscoor:RegistrationService>

<wsa:Address>

http://Business456.com/mycoordinationsservice/registration

</wsa:Address>

<wsa:ReferenceProperties>

<myApp:BetaMark> ... </myApp:BetaMark>
```

```

<myApp:EBDCode> ... </myApp:EBDCode>

</wsa:ReferenceProperties>

</wscoor:RegistrationService>

<myApp:IsolationLevel>
RepeatableRead
</myApp:IsolationLevel>

</wscoor:CoordinationContext>

...

</S:Header>

</S:Body>

...

</S:Body >

</S:Envelope>

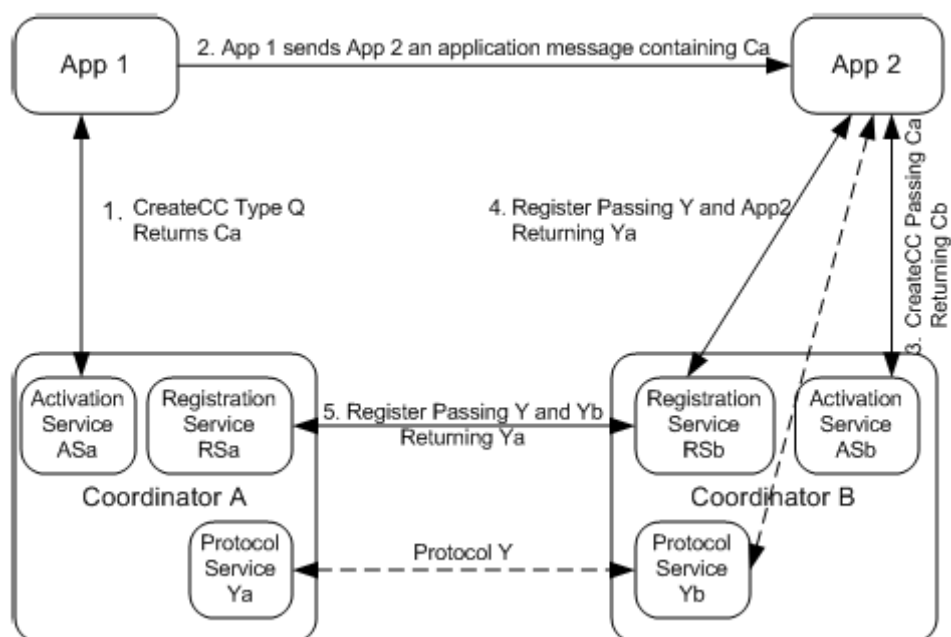
```

แอปพลิเคชัน ต้องการใช้ Activity ที่เหมาะสมจาก Coordination Service (Coordinator) แอปพลิเคชัน ต้องมีการรวมเอา Coordination Context เอาไว้ใน message ด้วย Context ถูกแลกเปลี่ยนกันในลักษณะของ SOAP header ฟิลด์ mustUnderstand จะต้องถูกเซทให้มีค่าเป็นจริง เพื่อให้ แอปพลิเคชัน ต้องทำการรับ SOAP header (เนื่องจาก SOAP header เป็นเพียงตัวเลือก แอปพลิเคชัน อาจจะไม่ต้องสนใจก็ได้ แต่ถ้ามีการเซท mustUnderstand ให้เป็นจริง แอปพลิเคชัน จะต้องรับ SOAP header)

5.2.4 Coordination Service (Coordinator)

Coordination Service (Coordinator) เป็นการประกอบไปด้วย Service ดังต่อไปนี้

- Activation Service มีการกำหนดให้ CreateCoordinationContext เป็นการสร้าง CoordinationContext โดย Coordination Service อาจจะรองรับ Activation Service
- Registration Service เป็นการกำหนดขั้นตอนวิธีการของการ Register ซึ่งมีส่วนในการอนุญาตให้ Web Service เข้ามาร่วมใน Coordination protocol โดย Coordination service ต้องรองรับ Registration Service
- เซทของ Coordination protocol สำหรับแต่ละ Coordination type ที่รองรับ ซึ่งทั้งหมดนี้จะถูกกำหนดในข้อกำหนดในส่วนของการ Coordination Type



รูปที่ 5.2 แอปพลิเคชัน และ Coordinator

จากรูปจะเป็นการแสดงการที่ แอปพลิเคชัน Services (App1 และ App2) กับ Coordinator ของตัวเอง (CoordinatorA และ CoordinatorB) ติดต่อกัน โดยมี Protocol Y และ Services Ya และ Yb เป็น Coordination Type ซึ่งไม่ได้ระบุใน WS-Coordination โดยจะมีขั้นตอนดังนี้

1. App1 ทำการส่ง CreateCoordinationContext สำหรับ Coordination Type Q โดยได้รับ Context Ca กลับมาซึ่งประกอบไปด้วยตัวระบุถึง Activity (Activity identifier) A1 โดยที่ Coordination type Q และมี Endpoint ที่อ้างอิงถึง Registration Service ของ CoordinatorA (RSa)
2. จากนั้น App1 จะส่ง แอปพลิเคชัน message ไปยัง App2 ซึ่งประกอบไปด้วย Context Ca
3. แต่เนื่องจาก App2 ต้องการใช้ CoordinatorB เพราะฉะนั้น App2 จึงใช้ CreateCoordinationContext โดยมี Ca เป็น input แล้วทำการเสนอให้กับ CoordinatorB จากนั้น CoordinatorB จึงทำการสร้าง CoordinationContext Cb ของมันเองซึ่งประกอบไปด้วยตัวระบุถึง Activity (Activity identifier) เช่นเดียวกัน และ Coordination Type เหมือนกับ Ca แต่เป็นการ Register เข้ากับ Service ของตัวมันเอง RSb
4. App2 ทำการพิจารณา Coordination protocol ที่รองรับของ Coordination type Q จากนั้นทำการ Register สำหรับ Coordination protocol Y ที่ CoordinatorB จากนั้นทำการเปลี่ยนจุด Endpoint ให้อ้างอิงถึง App2 และ Protocol Service Yb ซึ่งนี้จะทำให้เกิดการเชื่อมต่อเสมือนระหว่างจุด Endpoint ที่ Protocol Y สามารถใช้ได้

5. การ Register นี้จะทำให้ CoordinatorB ทำการส่งต่อการ register ไปยัง Registration Service ของ CoordinatorA (RSa) และทำการแลกเปลี่ยนจุด Endpoint สำหรับ Yb และ Protocol Service Ya ซึ่งจะทำให้เสมือนเกิดการเชื่อมต่อระหว่าง Endpoint ที่ protocol Y สามารถใช้ได้

Activation Service

สำหรับ Activation Service จะทำการสร้าง Activity ใหม่และคืนค่าเป็น Coordination Context ของมันกลับมา

แอปพลิเคชัน ส่ง CreateCoordinationContext จากนั้น Activation Service จะทำการคืนค่าของ CreateCoordinationContextRespond

- CreateCoordinationContext การร้องขออันนี้จะทำให้มีการสร้าง coordination context ที่มีการรองรับ coordination type (service ที่จัดหาเซทของ coordination protocol เช่น WS-AtomicTransaction) ซึ่งคำสั่งที่มีความจำเป็นเมื่อมีความต้องการที่จะเข้าถึงเครือข่ายที่มีสถานะแวดล้อมแตกต่างกัน (ต่าง platform กัน)

```
<CreateCoordinationContext ...>
<Expires> ... </Expires>? <CurrentContext> ... </CurrentContext>?
<CoordinationType> ... </CoordinationType>
...
</CreateCoordinationContext>
```

/CreateCoordinationContext/CoordinationType

ตัวบ่งบอกเฉพาะที่แสดงถึง coordination type ที่ต้องการสำหรับ activity (เช่น Atomic Transaction)

/CreateCoordinationContext/Expires

แสดงถึงเวลาที่หมดอายุโดยอยู่ในรูปของตัวเลขที่ไม่มีเครื่องหมายในหน่วย milliseconds ซึ่งอาจจะมีหรือไม่มีก็ได้

/CreateCoordinationContext/CurrentContext

อาจจะใช้ได้อย่างหลากหลายรวมไปถึงการ recovery และ subordinate ซึ่งอาจจะมีหรือไม่มีก็ได้

/CreateCoordinationContext /{any}

Element ที่อาจจะใช้สำหรับรายละเอียดเพิ่มเติม

/CreateCoordinationContext /@{any}

Attributes ที่อาจจะใช้สำหรับรายละเอียดเพิ่มเติม

ซึ่ง message CreateCoordinationContext อาจจะมีโครงสร้างง่ายๆดังนี้ก็ได้

```
<CreateCoordinationContext>
  <CoordinationType>
    http://schemas.xmlsoap.org/ws/2004/10/wsat
  </CoordinationType>
</CreateCoordinationContext>
```

- CreateCoordinationContextResponse จะเป็นการคืนค่า CoordinationContext ที่ถูกสร้างขึ้นมา โดยจะมีโครงสร้างดังนี้

```
<CreateCoordinationContextResponse ...>
  <CoordinationContext> ... </CoordinationContext>
  ...
</CreateCoordinationContextResponse>
```

/CreateCoordinationContext/CoordinationContext

ส่วนนี้คือ Coordination Context ที่ถูกสร้างขึ้นมา

/CreateCoordinationContext /{any}

Element ที่อาจจะใช้สำหรับรายละเอียดเพิ่มเติม

/CreateCoordinationContext /@{any}

Attributes ที่อาจจะใช้สำหรับรายละเอียดเพิ่มเติม

```
<CreateCoordinationContextResponse>
  <CoordinationContext>
    <Identifier>
      http://Business456.com/tm/context1234
    </Identifier>
    <CoordinationType>
      http://schemas.xmlsoap.org/ws/2004/10/wsat
```

```

</CoordinationType>

<RegistrationService>

<wsa:Address>

http://Business456.com/tm/registration

</wsa:Address>

<wsa:ReferenceProperties>

<myapp:PrivateInstance>

1234

</myapp:PrivateInstance>

</wsa:ReferenceProperties>

</RegistrationService>

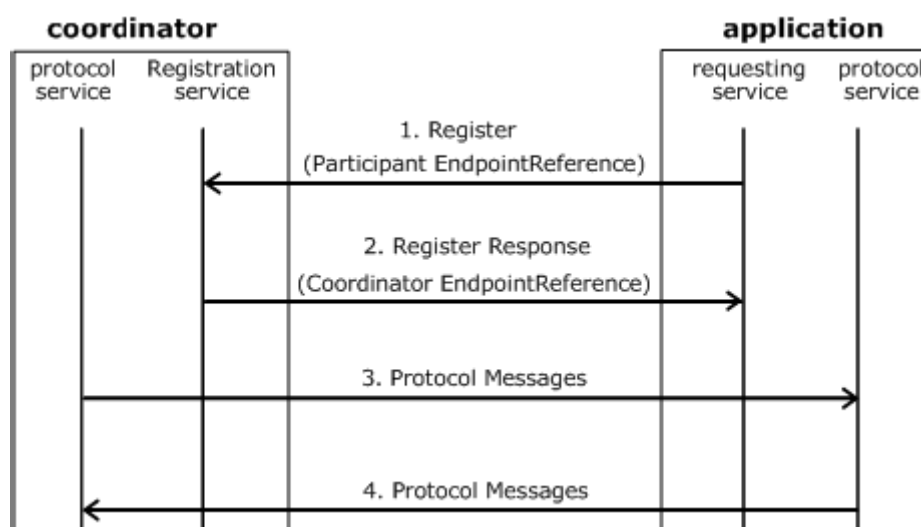
</CoordinationContext>

</CreateCoordinationContextResponse>

```

Registration Service

หลังจากที่ แอปพลิเคชัน มี coordination context จาก coordinator ที่ถูกเลือก แอปพลิเคชัน จะสามารถเข้าไป register สำหรับ Activity นั้นๆได้ โดย interface ที่ถูกจัดหาให้ สำหรับการ register ของ แอปพลิเคชัน เพื่อเข้าถึง activity และสำหรับ coordinator ในการ register เพื่อเข้าถึง activity นั้นเหมือนกัน



รูปที่ 5.3 การใช้จุดอ้างอิง Endpoint ระหว่างการลงทะเบียน

จากรูปที่ 5.3 coordinator จัดหาจุดอ้างอิง endpoint ในการ registration ให้ใน CoordinationContext ในขั้นตอน CreateCoordinationContext ซึ่ง service ที่ทำการร้องขอจุดอ้างอิง endpoint จะได้รับใน CoordinationContext ซึ่งอยู่ใน แอปพลิเคชัน message

1. register message จะทำการอ้างถึงจุดอ้างอิง endpoint นี้ และทำการรวมเอาจุดอ้างอิง endpoint ของ service participant protocol เอาไว้เป็น parameter
2. RegisterResponse ทำการรวมเอาจุดอ้างอิง endpoint ของ protocol service ของ coordinator
3. & 4. ในจุดนี้ทั้งสองฝั่งต่างมีจุดอ้างอิง endpoint ของ protocol service ของกันและกัน ดังนั้น protocol message สามารถ target ไปยังอีกฝั่งได้

จุดอ้างอิง endpoint เหล่านี้จะมี wsa:ReferenceProperties เป็นส่วนประกอบด้วยก็ได้ จากกฎของการ mapping ที่ถูกระบุไว้ใน WS-Addressing คุณสมบัติการอ้างอิงทั้งหมดต้องถูก copy เหมือนเป็น header ในทุกๆ message ที่ target ไปยัง endpoint

- Register Message
- RegistrationResponse Message

5.2.5 Coordination Fault

WS-Coordination Fault ต้องมีการ include URI:

<http://schemas.xmlsoap.org/ws/2004/10/wxcoor/fault>

โดยใน WS-Coordination จะมีการนิยามความผิดพลาดไว้โดยมีคุณสมบัติดังนี้

[Code] Fault code

[Subcode] Fault subcode

[Reason] ส่วนของเหตุผล

[Detail] รายละเอียดต่างๆ ถ้าว่างคือไม่มีรายละเอียดของความผิดพลาด

สำหรับ SOAP 1.2 [Code] ต้องเป็น Sender หรือ Receiver โดยคุณสมบัติเหล่านี้จะถูกทำให้เป็น XML ดังตาราง

SOAP Version	Sender	Receiver
SOAP 1.2	S:Sender	S:Receiver

ตาราง 5.1 คุณสมบัติของ [Code] สำหรับ SOAP 1.2

โดยสามารถนำเอาคุณสมบัติในตารางข้างต้นมาเชื่อมเป็น SOAP 1.2 ได้ดังนี้

```
<S:Envelope>
<S:Header>
<wsa:Action>
http://schemas.xmlsoap.org/ws/2004/10/wscoor/fault
</wsa:Action>
<!-- Headers elided for clarity. -->
</S:Header>
<S:Body>
<S:Fault>
<S:Code>
<S:Value>[Code]</S:Value>
<S:Subcode>
<S:Value>[Subcode]</S:Value>
</S:Subcode>
</S:Code>
<S:Reason>
<S:Text xml:lang="en">[Reason]</S:Text>
</S:Reason>
<S:Detail>
[Detail]
...
</S:Detail>
</S:Fault>
</S:Body>
</S:Envelope>
```

Invalid State

สำหรับความผิดพลาดประเภทนี้จะถูกส่งได้ทั้ง Coordinator หรือ Participant เพื่อจะกำหนดจุด Endpoint ที่ทำให้เกิดความผิดพลาดนั้น ซึ่งได้รับ *message* ที่ไม่ *valid* สำหรับสถานะตอนนั้นของมัน ซึ่งสถานการณ์นี้ไม่สามารถทำการกู้คืนได้

คุณสมบัติ

[Code] Sender

[Subcode] wscoor:InvalidState

[Reason] The message was invalid for the current state of the activity.

[Detail] unspecified

Invalid Protocol

สำหรับความผิดพลาดประเภทนี้จะถูกส่งได้ทั้ง Coordinator หรือ Participant เพื่อจะกำหนดจุด Endpoint ที่ทำให้เกิดความผิดพลาดนั้น ซึ่งได้รับ *message* ที่ถูกส่งมาด้วย *Protocol* ที่ไม่ถูกต้อง ซึ่งสถานการณ์นี้ไม่สามารถทำการกู้คืนได้

คุณสมบัติ

[Code] Sender

[Subcode] wscoor:InvalidProtocol

[Reason] The protocol is invalid or is not supported by the coordinator

[Detail] unspecified

Invalid Parameters

สำหรับความผิดพลาดประเภทนี้จะถูกส่งได้ทั้ง Coordinator หรือ Participant เพื่อจะกำหนดจุด Endpoint ที่ทำให้เกิดความผิดพลาดนั้น ซึ่งได้รับ *message* ที่มี *parameter* ที่ *invalid* ซึ่งสถานการณ์นี้ไม่สามารถทำการกู้คืนได้

คุณสมบัติ

[Code] Sender

[Subcode] wscoor:InvalidParameters

[Reason] The message contained invalid parameters and could not be processed

[Detail] unspecified

No Activity

สำหรับความผิดพลาดประเภทนี้จะถูกส่งได้จาก Coordinator ถ้า participant ไม่มีการติดต่อมาเป็นระยะเวลาที่นานเกินไป

คุณสมบัติ

[Code] Sender

[Subcode] wscoor:ContextRefused

[Reason] The coordination context that was provided could not be accepted

[Detail] unspecified

Context Refused

สำหรับความผิดพลาดประเภทนี้จะถูกส่งไปยัง Coordinator เพื่อจะกำหนดจุด Endpoint ที่ไม่สามารถทำการรับ Context ที่ถูกส่งมาได้

คุณสมบัติ

[Code] Sender

[Subcode] wscoor:ContextRefused

[Reason] The coordination context that was provided could not be accepted

[Detail] unspecified

Already Registered

สำหรับความผิดพลาดประเภทนี้จะถูกส่งไปยัง participant ถ้า coordinator ตรวจพบว่า participant พยายามที่จะ Register สำหรับ protocol ตัวเดียวกันให้กับ activity ตัวเดียวกันมากกว่า 1 ครั้ง

คุณสมบัติ

[Code] Sender

[Subcode] wscoor:AlreadyRegistered

[Reason] The participant has already registered for the same protocol

[Detail] unspecified

5.2.6 Security Model

จุดประสงค์หลักของ Security โดยอ้างอิงถึง WS-Coordination คือ

- ให้แน่ใจว่ามีเพียงขั้นตอนที่ได้รับการอนุมัติแล้วทำการสร้าง Coordination context
- ให้แน่ใจว่ามีเพียงขั้นตอนที่ได้รับการอนุมัติแล้วทำการ Register เข้ากับ Activity

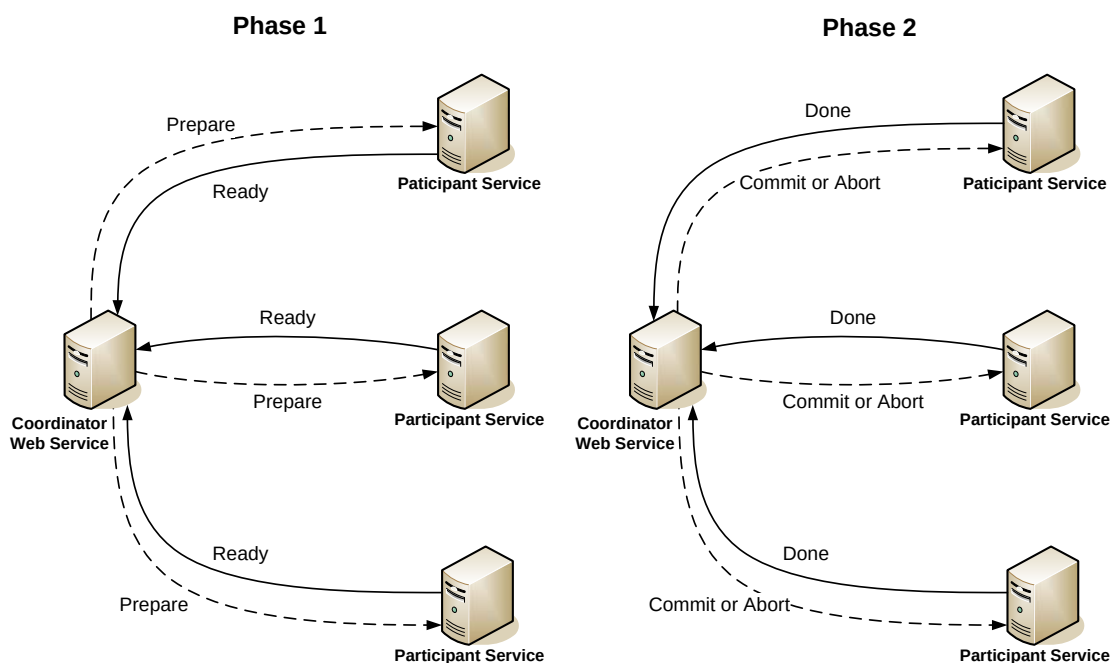
- ให้แน่ใจว่ามีเพียง Coordination Context ที่ถูกต้องตามกฎหมายทำการ Register
- ทำให้การทำงานของ Security ที่มีอยู่มีประสิทธิภาพมากขึ้น
- อนุญาตให้มีการจำกัดสิทธิ์โดยอ้างอิงถึง federated identities

จุดประสงค์หลักเหล่านี้ถูกกำหนดขึ้นมาจากความต้องการของ Security เพื่อ ความถูกต้องของข้อมูล การรักษาความลับ และการแสดงความเป็นเจ้าของ ซึ่งความต้องการแต่ละอย่างจะถูกจัดทำให้โดยองค์กรที่สร้างข้อกำหนดเกี่ยวกับ Web Service Security เช่น WS-Security และ WS-Trust

5.3 WS-AtomicTransaction (WS-AT)

Specification นี้จะให้คำจำกัดความของ atomic transaction ซึ่งเป็นรูปแบบของการประสานงานกันของระบบแบบ distributed ชนิดหนึ่งโดยจะใช้กับ framework ที่เราได้พูดถึงไปใน specification ของ WS-Coordination โดย specification นี้ทำการให้นิยามของ protocol ที่จะทำการประสานการเป็นไปอย่างสอดคล้องที่จะถูกใช้โดยการประสานงานแบบ atomic transaction ไว้ 3 ชนิดด้วยกันคือ

- Completion
- Volatile two-phase commit
- Durable two-phase commit



รูปที่ 5.4 การทำงานของ Two-phase commit protocol

ซึ่งเป็น Protocol ที่ WS-Atomic Transaction ใช้ในการทำ Transaction บน Web Services

โดยผู้พัฒนาระบบจะสามารถใช้ Protocol ใดก็ได้ใน 3 ชนิดนี้ในการสร้าง application ซึ่งต้องการความสอดคล้องกันของผลลัพธ์สำหรับกิจกรรมบนระบบแบบ distributed ที่มีอายุสั้นๆ ที่จะต้องมีคุณสมบัติ **“ทำสำเร็จทั้งหมดหรือว่าไม่ทำเลย”**

5.3.1 การcommit แบบสองระยะ (Two-Phase Commit Protocol)

การCommit แบบสองระยะเป็นกลไกที่จะช่วยให้การทำงานของแอปพลิเคชันแบบกระจายสามารถทำการ Commit Transaction ร่วมกันได้ และถ้ามีความผิดพลาดใดๆเกิดขึ้น Transaction นั้นๆจะต้องเหมือนกับไม่เคยเกิดขึ้นมาก่อน (Rollback) ซึ่งเป็นการช่วยให้คงความถูกต้องของข้อมูล การCommit แบบสองระยะจะประกอบไปด้ว้การทำงานสองระยะ ดังนี้

ระยะที่ 1

1. Coordinator ทำการส่ง <Prepare T> ไปให้กับเซอร์วิสที่เกี่ยวข้องใน Transaction ทั้งหมด
2. จากนั้นเซอร์วิสจะทำการดูว่างานที่ได้รับมานั้นสามารถทำงานได้สำเร็จหรือไม่
 - a. ถ้าสำเร็จเซอร์วิสจะทำการส่ง <Ready T> ไปให้กับ Coordinator
 - b. ถ้าไม่สำเร็จเซอร์วิสจะทำการส่ง <Abort T> ไปให้กับ Coordinator
3. Coordinator รอจนกระทั่งได้รับเมสเซจจากทุกๆเซอร์วิสครบแล้ว

ระยะที่ 2

ถ้าทุกๆเมสเซจที่ Coordinator ได้รับคือ <Ready T> แล้ว

1. Coordinator ทำการเขียน <Commit> ลงในไฟล์บันทึก
2. Coordinator ทำการส่ง <Commit T> ไปให้กับเซอร์วิสทุกตัวที่เกี่ยวข้องใน Transaction
3. เซอร์วิสที่เกี่ยวข้องทุกตัวทำการ Commit งานของตัวเองจากนั้นทำการคืนทรัพยากรทั้งหมดที่ใช้ให้กับระบบ
4. เซอร์วิสทุกตัวทำการตอบรับการ Commit ไปยัง Coordinator
5. Coordinator สิ้นสุดการทำงานใน Transaction

ถ้า Coordinator ได้รับ <Abort T> จากเซอร์วิสตัวใดตัวหนึ่ง

1. Coordinator ทำการเขียน <Rollback> ลงในไฟล์บันทึก
2. Coordinator ทำการส่ง <Rollback T> ไปให้กับเซอร์วิสทุกตัวที่เกี่ยวข้องใน Transaction
3. เมื่อเซอร์วิสได้รับ <Rollback T> จะทำการ Rollback งานที่ทำของตน แล้วตอบรับกลับไปยัง Coordinator
4. Coordinator สิ้นสุดการทำงานใน Transaction

5.3.2 Atomic Transaction Context

Atomic Transaction สร้างขึ้นมาเพื่อทำการอธิบายถึง Coordination Type ที่ได้รับการกำหนดไว้บน WS-Coordination

Atomic Transaction Coordination Context ต้องส่งผ่านอยู่ในทุก แอปพลิเคชัน Message ที่เกี่ยวข้องกับ Transaction นอกจากนั้นยังมีการเพิ่มความหมายของขั้นตอน

CreateCoordinationContext ในส่วนของ Activation Service

- ถ้า request มีการรวมเอาส่วนของ CurrentContext Coordinator จะทำการแทรกเข้าไปเหมือนกับ subordinate ให้กับ coordinator ใน CurrentContext
- ถ้า request ไม่มีการรวมเอาส่วนของ CurrentContext Coordinator จะทำการสร้าง Transaction ใหม่ที่จะทำหน้าที่เหมือนเป็น root

Coordination Context อาจจะมี Expires attribute ประกอบอยู่ด้วยซึ่งเป็นการกำหนดเวลาที่เร็วที่สุดที่ Transaction อาจจะถูก terminate ได้จากการใช้เวลานานเมื่อถึงจุดนั้นแล้ว Transaction Manager อาจจะมีการทำการเลือกกว่าจะทำการ roll back transaction นั้นหรือไม่ ถ้ายังไม่มีการ Transmit , Commit หรือ Prepare notification

Atomic Transaction protocol ถูกระบุโดย Coordination Type:

<http://schemas.xmlsoap.org/ws/2004/10/wsat>

5.3.3 Atomic Transaction Protocols

ในส่วนของ WS-AtomicTransaction จะทำการระบุถึง Protocol ต่อไปนี้สำหรับการทำ Atomic Transaction

- Completion: protocol นี้จะทำการเริ่มต้นกระบวนการในการ commit โดยอ้างจากแต่ละ protocol ที่ถูก register โดย participants ซึ่ง Coordinator จะเริ่มต้นด้วย Volatile 2PC จากนั้นจะเข้าไปสู่ Durable 2PC โดยผลลัพธ์สุดท้ายจะถูกส่งให้กับ initiator
- Two-Phase Commit (2PC): 2PC Protocol จะกำหนดให้ participant จะทำการ commit หรือว่า abort และให้แน่ใจว่า participant ทุกตัวได้รับการแจ้งผลลัพธ์สุดท้าย โดย 2PC protocol ประกอบไปด้วย
 - Volatile 2PC: Participant ที่มีการจัดการกับ Volatile resource เช่น Cache ควรจะทำการ register เข้ากับ Protocol
 - Durable 2PC: Participant ที่มีการจัดการกับ Durable resource เช่น Database ควรจะทำการ register เข้ากับ Protocol

Participant สามารถทำการ register กับ Protocol ได้มากกว่า 1 protocol ซึ่งทำได้โดยการส่ง Register message หลายๆ message

5.3.4 Preconditions

ในการที่ Protocol จะทำงานได้ตามขั้นตอนอย่างถูกต้องจำเป็นต้องมีการทำงานก่อนที่จะเริ่มต้นการประมวลผลดังนี้

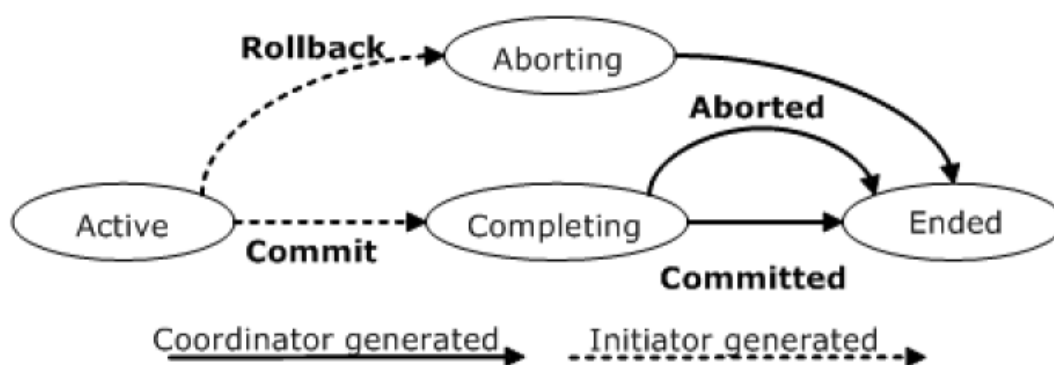
1. Source ต้องรู้ว่ากฎของ destination เป็นอย่างไร และถ้ามี source ต้องมีความสามารถในการที่จะสร้าง message ให้สอดคล้องตามกฎของ destination
2. ถ้ามีความต้องการของ secure exchange of message ดังนั้น source และ destination ต้องมี Security Context

5.3.5 Completion Protocol

Completion Protocol ถูกใช้โดย แอปพลิเคชัน เพื่อทำการบอก coordinator ว่าจะพยายาม Commit หรือจะ Abort Atomic Transaction หลังจาก transaction นั้นทำงานจนเสร็จแล้วสถานะของมันจะถูกส่งกลับมาให้ยัง แอปพลิเคชัน

Initiator register ของ Protocol นี้ใช้ Protocol identifier ดังนี้:

<http://schemas.xmlsoap.org/ws/2004/10/wsac/Completion>



รูปที่ 5.5 ภาพรวมของ Completion Protocol

Coordinator accepts:

- Commit - หลังจากที่ได้รับแจ้งมาทำให้ Coordinator รู้ว่า Participant มีความสามารถในการที่จะทำงานให้เสร็จ และมันควรพยายามที่จะ Commit Transaction นั้น
- Rollback - หลังจากที่ได้รับแจ้งมา Coordinator รู้ว่า Participant ได้ทำการ terminate การประมวลผลของการทำงานใน แอปพลิเคชัน นั้น และมันควรที่จะทำการ Abort Transaction

Initiator accepts:

- Committed - หลังจากที่ได้รับการแจ้งมา Initiator รู้ว่า coordinator ถึงเวลาที่ต้องทำการ Commit transaction
- Aborted - หลังจากที่ได้รับการแจ้งมา Initiator รู้ว่า Coordinator ถึงเวลาที่ต้องทำการ abort transaction

5.3.6 ลักษณะของ Two-Phase Commit Protocol ใน WS-AtomicTransaction

Two-Phase Commit Protocol (2PC) เป็น Coordination Protocol ที่ทำการกำหนดว่า Participants หลายตัวจะทำงานร่วมกันตามข้อตกลงของผลลัพธ์ใน Atomic Transaction ได้อย่างไร 2PC Protocol แบ่งออกได้เป็น Volatile 2PC และ Durable 2PC

■ Volatile Two-Phase Commit Protocol

หลังจากที่ได้รับการแจ้ง Commit มาในส่วนของ Completion Protocol จากนั้น Root Coordinator จะทำการเตรียมพร้อมให้ทุกๆ participant ทำการ register

สำหรับ Volatile 2PC Protocol ทุกๆ Participant ที่ถูก register เข้า Protocol (Volatile 2PC) นี้จะต้องทำการ respond ก่อนที่จะ Register เข้ากับ Durable 2PC

Volatile Recipient จะไม่มีการรับรองว่าจะได้รับการแจ้งเมื่อมีผลลัพธ์ของ Transaction

Participant ที่ register กับ Volatile 2PC สามารถใช้ Protocol identifier ดังนี้ :

<http://schemas.xmlsoap.org/ws/2004/10/wsat/Volatile2PC>

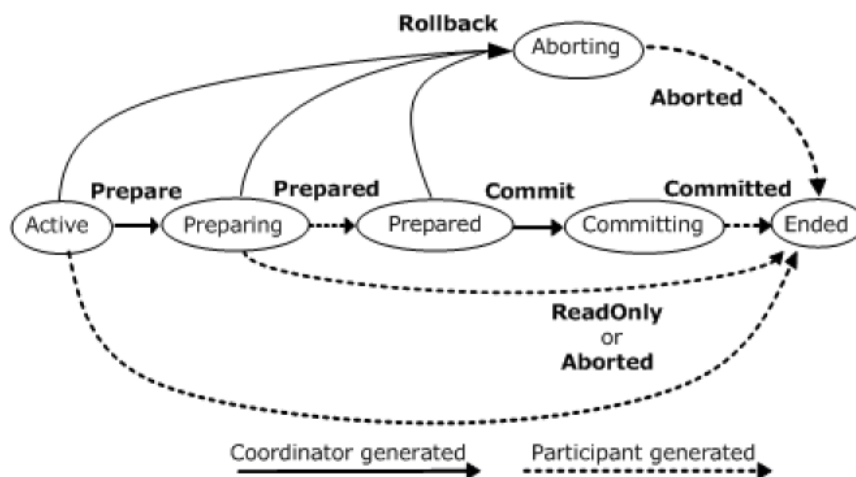
■ Durable Two-Phase Commit Protocol

หลังจากที่ได้รับการแจ้ง Commit ในส่วนของ Completion Protocol และเสร็จสิ้นจากการเตรียมสถานะของ Volatile 2PC แล้ว Root Coordinator จะเริ่มทำการเตรียมสำหรับให้ Participant Register เข้าสู่ Durable 2PC Protocol โดยจะต้องมีการ respond ว่า Prepare หรือ ReadOnly ก่อนที่จะมีการแจ้ง Commit ให้กับ participant เพื่อให้มีการ register กับ protocol ตัวใดตัวหนึ่ง

Participant ที่ register กับ Durable 2PC สามารถใช้ Protocol identifier ดังนี้ :

<http://schemas.xmlsoap.org/ws/2004/10/wsat/Durable2PC>

■ 2PC Diagram and notifications



รูปที่ 5.6 ภาพรวมของการ Commit แบบสองระยะ

Participant accepts:

- Prepare

หลังจากที่ได้รับการแจ้งมา Participant จะรู้ว่าต้องเข้าสู่ Phase 1 และทำการ vote ให้กับผลลัพธ์ของ Transaction ถ้า Participant ไม่รู้เกี่ยวกับ Transaction มันต้องทำการ abort แต่ถ้า Participant นั้นได้ทำการ vote ไปแล้วมันควรจะทำการ resend ค่าที่เคย vote ให้

- Rollback

หลังจากที่ได้รับการแจ้งมา Participant จะรู้ว่าต้องทำการ abort และ ทำการทิ้ง Transaction นั้นไปการแจ้งนี้สามารถส่งได้ทั้ง phase 1 และ phase 2 เมื่อมีการส่งแล้ว coordinator จะไม่สนใจ และ clear ที่ทั้งหมดของ Transaction

- Commit

หลังจากที่ได้รับการแจ้งมา Participant จะรู้ว่าต้องทำการ commit Transaction โดยการแจ้งนี้จะสามารถส่งได้หลังจาก phase 1 และ Participant ทำการ vote commit ถ้า participant ไม่รู้เกี่ยวกับ Transaction มันต้องทำการส่งการแจ้งว่า committed ให้กับ Coordinator

Coordinator accepts:

- Prepared

หลังจากที่ได้รับการแจ้งมา Coordinator จะรู้ว่า participant ได้ถูกเตรียม และทำการ commit Transaction

- ReadOnly

หลังจากที่ได้รับการแจ้งมา Coordinator จะรู้ว่า participant ได้ทำการ commit Transaction แล้ว และไม่สนใจใน Transaction อีก Participant ไม่ต้องการที่จะเข้ามามีส่วนร่วมใน phase 2

- Aborted

หลังจากที่ได้รับการแจ้งมา Coordinator จะรู้ว่า participant ได้ทำการ abort แล้ว และไม่สนใจใน Transaction อีก

- Committed

หลังจากที่ได้รับการแจ้งมา Coordinator จะรู้ว่า Participant ได้ทำการ committed transaction แล้วทำให้ Participant สามารถไม่ต้องสนใจกับ Transaction อีก

- Replay

หลังจากที่ได้รับการแจ้งมา Coordinator จะทำการสรุปได้ว่า Participant มีความผิดพลาดขึ้นซึ่งไม่สามารถทำการกู้คืนได้ Coordinator จึงควรที่จะ resend Protocol ที่เหมาะสมที่สุด

5.3.7 Transaction Faults

WS-AtomicTransaction fault ต้องมีการ include เอา [action] จาก URI:

<http://schemas.xmlsoap.org/ws/2004/10/wsat/fault>

Faults จะเน้นไปที่ destination endpoint ซึ่งเกี่ยวข้องกับกฎของการรองรับการเกิด faults ซึ่งได้รับการระบุไว้ใน WS-Addressing

Faults ในส่วนนี้จะถูกระบุไว้ด้วยคุณสมบัติดังนี้

[Code] Fault code

[Subcode] Fault subcode

[Reason] เหตุผลของการเกิด fault (ภาษาอังกฤษ)

[Detail] รายละเอียดต่างๆ ถ้าว่างคือไม่มีรายละเอียดของความผิดพลาด

โดยสามารถนำเอาคุณสมบัติในตารางข้างต้นมาเชื่อมเป็น SOAP 1.2 ได้ดังนี้

```
<S:Envelope>
```

```
<S:Header>
```

```
<wsa:Action>
```

```
http://schemas.xmlsoap.org/ws/2004/10/wsat/fault
```

```

</wsa:Action>

<!-- Headers elided for clarity. -->

</S:Header>

<S:Body>

<S:Fault>

<S:Code>

<S:Value>[Code]</S:Value>

<S:Subcode>

<S:Value>[Subcode]</S:Value>

</S:Subcode>

</S:Code>

<S:Reason>

<S:Text xml:lang="en">[Reason]</S:Text>

</S:Reason>

<S:Detail>

[Detail]

...

</S:Detail>

</S:Fault>

</S:Body>

</S:Envelope>

```

InconsistentInternalState

Fault ประเภทนี้จะถูกส่งโดย Participant เพื่อทำการบอกว่าไม่สามารถทำตามวัตถุประสงค์ซึ่งจะเป็นการบอกถึง Global consistency failure และเป็นสถานะที่ไม่สามารถทำการกู้คืนได้

[Code] Sender

[Subcode] wsat:InconsistentInternalState

[Reason] A global consistency failure has occurred. This is an unrecoverable condition.

[Detail] unspecified

5.4 ความสามารถในการสนับสนุน WS-AtomicTransaction บน WebSphere

Application Server

WebSphere Application Server มีการนำ Web Services Atomic Transaction (WS-AT) มาใช้ โดย specification นี้จะทำให้ Web service สามารถที่จะมีส่วนร่วมใน global transaction ซึ่งถูกทำให้กระจายไปยังสภาพแวดล้อมของ Web Services ที่ต่างกัน

- Web Services Atomic Transaction สำหรับ WebSphere Application Server ให้บริการเกี่ยวกับคุณภาพของบริการทางด้าน transaction บนสภาพแวดล้อมของ Web Services ซึ่งจะทำให้ Web Services ที่อยู่บนระบบแบบ distributed และทรัพยากรที่ใช้นั้นสามารถเป็นส่วนหนึ่งของ global transaction
- ความสามารถในการใช้งาน WS-AT ที่สนับสนุนนั้นเป็นรูปแบบที่สามารถใช้งานข้ามระบบได้โดยไม่ต้องมีการเขียนโปรแกรมติดต่อเพิ่มเติมโดยการกำหนดเขตของ Global transaction นั้นจะถูกจัดการโดยมาตรฐานบน J2EE ซึ่งใช้ JTA UserTransaction โดยถ้ามีการเรียกใช้งาน Web Services จาก application ที่ทำงานอยู่ภายใต้ global transaction สภาพแวดล้อมของ WS-AT Coordinator จะทำการส่งคำร้องนั้นไปยัง Web Service เป้าหมาย

ถ้า WebSphere Application Server นั้นเป็นเจ้าของระบบที่รองรับการร้องขอที่อยู่ภายใต้สภาพแวดล้อมของ WS-AT แล้ว WebSphere จะทำการเรียกใช้งานและสร้าง JTA transaction ในระบบปลายทางที่กำลังจะกลายเป็นสภาพแวดล้อมของ Transaction โดยอัตโนมัติ

5.4.1 การตั้งค่า Deployment descriptors สำหรับ atomic transactions

ในการเปิดใช้งาน WS-AT จะต้องมีการปรับปรุง deployment descriptors ทั้งในส่วน of Web และ EJB ดังนี้

- ในส่วนของเว็บที่เรียกใช้งาน Web Services นั้นจะต้องมีการกำหนดค่า Send Web Services Atomic Transaction on requests เพื่อการใช้งาน Transaction บน Web Service ปลายทาง
- ในส่วนของเว็บที่เป็น Web Service นั้นจะต้องมีการกำหนดค่า Execute using Web Services Atomic Transaction on incoming requests เพื่อให้ Web Service นั้นวิ่งอยู่บนสภาพแวดล้อมของ Transaction

- ในส่วนของ EJB ที่เรียกใช้งาน Web Service นั้นกำหนดค่า Use Web Services Atomic Transaction เพื่อกำหนดให้ใช้งาน Atomic Transaction บน Service ปลายทาง
- ในส่วนของ EJB นั้นจำเป็นต้องมีการกำหนดค่าชนิดของ Transaction เป็น Required ซึ่งเป็นค่ามาตรฐานเพื่อการใช้งาน Atomic Transaction